Response to Reviewer 3

Enhancing Consent Policy Privacy in Blockchain-Based Healthcare Data Sharing

---

Comments 1: *The trust model around the Trusted Proof Service is a major limitation and should be discussed more critically. The paper explicitly assumes that the TPS preserves policy material and requester evidence, but the current evaluation even co-locates the TPS with the provider. This weakens the practical privacy claim, since the most sensitive information is moved off-chain rather than cryptographically protected from the service itself. The paper should more clearly distinguish what is protected from public-chain observers and what is merely trusted to the TPS.*

Response 1: We agree that this is the main architectural limitation, and the revision states it as such in the Problem, Discussion, and Conclusion sections. The distinction the reviewer asks for is now made in two places. The trust-model subsection separates authorization correctness, which the commitment and the on-chain verification enforce regardless of the TPS (and which the revised relation strengthens by binding the program length, the authorization context, and the issuer-signed claims), from policy confidentiality, which is not cryptographically enforced against the TPS and rests on the service following its workflow; it also names what a compromised or colluding TPS can do (expose policy and evidence, substitute unsigned evidence, refuse to prove) and notes that the provider-operated TPS of the evaluation concentrates that trust at the provider. A new threat/guarantee table then distinguishes the public-chain observer, untrusted requester, workflow-following TPS, compromised or colluding TPS, forged credentials, and unauthorized or revoked issuers across policy confidentiality, authorization correctness, liveness, and decision auditability. A note below the table states that the provider-operated deployment used in the evaluation concentrates every TPS-dependent assumption at the provider.

The claim is scoped accordingly: what the evaluation establishes is what a public-chain observer can infer from the artifacts a workflow-following TPS produces, and the abstract, Discussion, and Conclusion say that confidentiality against the service itself is a trust assumption, not a cryptographic property. The Discussion also names the mechanisms that would relocate that trust—collaborative-SNARK or attested proof generation so that no single party sees both policy and evidence, a holder-signed authorization context so that the service cannot reuse evidence it has seen, and accountable proof obligations so that refusals become attributable—and the Related Work section now relates this to self-sovereign-identity designs with holder-authorized presentation (Comment 5).

Changes in the manuscript. The trust distinction is stated explicitly in the system model, summarized in a new threat/guarantee matrix, interpreted as the principal architectural trade-off in the Discussion, and carried into the limitations and future directions of the Conclusion.

Location in the revised manuscript: Section "Problem and Threat Model," subsection "System and Trust Model," paragraphs on the provider-operated TPS.

> For implementation and evaluation simplicity, the current evaluation co-locates the TPS with the provider (a provider-operated TPS). This co-location concentrates trust at the provider side: a single party now holds both the policy and the requester evidence, so a compromised provider-operated TPS could substitute inputs whose origin is not independently authenticated or withhold proof generation from

legitimate requesters. Two guarantees must be distinguished here. Authorization correctness is primarily enforced by the commitment and on-chain proof verification: a proof that does not satisfy the committed policy relation is rejected, largely independently of how the TPS is instantiated. Policy confidentiality, in contrast, is not cryptographically enforced against the TPS itself; it rests entirely on the trust assumption stated above. Our evaluation therefore assumes a TPS that follows the prescribed workflow and isolates the remaining question: what a public-chain observer can infer from the resulting public artifacts. Weakening the TPS trust assumption itself is orthogonal and is the role of the reduced-trust instantiations above. Table 3 summarizes, for each actor or failure mode, which of the four guarantees—policy confidentiality, authorization correctness, liveness, and decision auditability—is enforced cryptographically, which holds only under the TPS trust assumption, and which the prototype leaves to external functions or does not provide.

Location in the revised manuscript: Section "Problem and Threat Model," new Table 3, "Guarantees under each actor or failure mode."

Table 3. Guarantees under each actor or failure mode.

Actor or failure mode	Policy confidentiality	Authorization correctness	Liveness	Decision auditability
Passive public-chain observer	C for proof and commitment contents; empirical for modeled metadata	C	n/a	C: outcome recorded
Untrusted requester	C against disclosure from public artifacts; adaptive-query leakage not evaluated	C: cannot forge a proof or alter its bound authorization context	n/a	C: pseudonymous request context and outcome recorded on-chain
Workflow-following TPS	– against the TPS itself; T against onward disclosure	C for the committed relation; T/E for unsigned evidence provenance	T	C for submitted decisions; – for refusal
Compromised or colluding TPS	–	C for the committed relation and signed claims; E for externally authenticated evidence; – for unauthenticated evidence provenance	–: may refuse to prove	C for submitted decisions; – for refusal
Forged credential	n/a	C: signature verified under the policy-bound issuer key	n/a	n/a
Unauthorized or revoked issuer	n/a	E: issuer authorization and revocation status are external	n/a	C for the on-chain decision record; E for determining issuer status

Note: C: cryptographic; T: trust in the TPS following the prescribed workflow; E: external or lifecycle function; –: not provided by the prototype; n/a: not applicable. In the evaluated provider-operated deployment, the provider and TPS are co-located, so all TPS-dependent confidentiality and liveness assumptions are concentrated at the provider.

Location in the revised manuscript: Section "Discussion," paragraphs on the TPS trust trade-off and reduced-trust directions.

The TPS trust assumption is the main architectural tradeoff. Moving confidentiality-sensitive proof generation to the TPS keeps policy-bound proving information away from requesters and public smart contracts while preserving on-chain verifiability,

> but it shifts part of the confidentiality burden to the service layer (Section 4). This tradeoff leaves on-chain verifiability intact but does affect liveness and accountability: the TPS can refuse to produce a proof, and such a refusal may be indistinguishable from a legitimate denial at the on-chain interface.
>
> Three directions follow. First, adding an explicit chain/contract domain and an authenticated submission channel would extend context binding beyond a single deployment and reduce front-running risk. Second, accountable proof obligations—for example, on-chain commitments to serve requests together with timeout-with-evidence mechanisms—would make proof refusals attributable. Third, the reduced-trust TPS instantiations of Section 4 (collaborative-SNARK or TEE-backed proving) would relax the trust concentration itself, including the attribute-origin limitation; realizing and evaluating them is left to future work.

Location in the revised manuscript: Section "Conclusion," limitations and future-directions paragraphs.

> These findings are empirical and scoped to passive observation of commitments, proofs, public parameters, calldata, events, and gas on the generated-policy corpus. Policy confidentiality remains contingent on the TPS following its workflow, and institutional credential trust remains contingent on issuer governance. The proved policy class covers access-time checks for bounded Boolean policies over standardized terms. Outside this scope lie downstream purpose limitation, post-access duties, delegation, credential revocation, and channels involving timing, ordering, mempool activity, off-chain communication, adaptive probing, or correlations between authorization outcomes and policy classes.
>
> The next steps are therefore clear: reduced-trust proof generation and holder-signed request contexts can reduce the TPS trust concentration; governed issuer registries with in-circuit membership and revocation witnesses can strengthen credential provenance; and evaluation against active adversaries can test transcript normalization in the timing and ordering channels beyond the present model.

Comments 2: *The current proof statement leaves important soundness gaps. The paper acknowledges that the program length is not bound into the policy commitment and that the proof is not bound to the requester, resource, or session context. These are not minor implementation details: they directly affect the correctness and replay-resistance of the authorization workflow. These issues should be fixed in the design and experimentally re-evaluated, rather than mainly left to future work.*

Response 2: Both gaps are closed in the revised design and implementation, and the evaluation was repeated on the revised system.

The policy commitment now covers the program length, $h = \mathsf{Poseidon}(\mathsf{H}_{\mathrm{pol}}(P_{\mathrm{enc}}) \parallel \ell \parallel r)$, and the same private ℓ drives both the commitment check and the interpreter, so an authorizing proof cannot rest on a shorter stack-consistent prefix. The proof statement now carries a public context digest $d_\chi = \mathsf{Poseidon}(id_{\mathrm{req}} \parallel id_{\mathrm{res}} \parallel \nu)$ that the circuit constrains and the access-control contract recomputes from the transaction parameters, so a captured proof cannot be paired with a fresh nonce, another resource, or another requester within a deployment. The measured marginal costs (88 constraints for the length binding; 605 constraints, one public input, and about 56,000 gas for the context binding including the on-chain Poseidon recomputation) are given in the responses to Reviewer 1, Comments 1 and 2, which also describe the regression and end-to-end tests.

The re-evaluation is complete rather than partial. All of-record numbers in the revised manuscript come from a new capture of 1,077 policies and 1,773 Polygon Amoy executions under the frozen revised release (five-signal statement, length- and context-bound relation, credential-derived profile attributes), followed by a new un-padded ablation run of 1,119 executions on the same policy subset. The cost profile reflects the revised circuit (81,740 constraints) and the revised on-chain path, and the results sections report the outcome of the privacy attacks on the revised transcript, including two criterion failures that we report and trace to the padding's random draws rather than to the policy (Reviewer 1, Comments 5 and 6).

Changes in the manuscript. The Design section now defines both bindings as part of the proved relation. The Privacy Evaluation was regenerated from the revised implementation; the updated corpus description, headline results, and final cost profile are reproduced below, with the detailed PCH, PI, SPP, clustering, and ablation tables also replaced by the re-run results.

Location in the revised manuscript: Section "TPS-ZKPI Design," subsection "Commitment Binding and Proof Generation," policy-length binding.

> To bind each proof to a provider-defined hidden policy, the provider samples commitment randomness r uniformly from the scalar field and computes
>
> $$q = \mathsf{H}_{\mathrm{pol}}(P_{\mathrm{enc}}),$$
> $$h = \mathsf{Com}(P_{\mathrm{enc}}, \ell; r) = \mathsf{Poseidon}(q \parallel \ell \parallel r),$$
>
> where q is the Poseidon-based digest of the fixed-length encoded policy, ℓ is the private program length, r is the commitment randomness, and h is the policy commitment stored on-chain. The semantics of an encoded policy are determined by the pair (P_{enc}, ℓ), since ℓ identifies the active prefix of the padded token array. Accordingly, the circuit supplies the same private value ℓ to both the commitment verifier and the policy interpreter. Under the collision resistance of Poseidon, an accepting proof bound to h cannot substitute a different program length and therefore cannot evaluate an alternative stack-consistent prefix of P_{enc}.

Location in the revised manuscript: Section "TPS-ZKPI Design," subsection "Commitment Binding and Proof Generation," authorization-context binding.

> For an access request, let
>
> $$\chi = (id_{\mathrm{req}}, id_{\mathrm{res}}, \nu), \qquad d_\chi = \mathsf{Poseidon}(id_{\mathrm{req}} \parallel id_{\mathrm{res}} \parallel \nu),$$
>
> where ν is a fresh session nonce. The context tuple χ consists of private circuit inputs, while its digest d_χ is public. The current public-signal vector is
>
> $$(1, h, pp, d_\chi, \tau),$$
>
> where pp is a fixed-format commitment to the per-predicate parameters and τ is the request epoch used by the temporal checks. Although the three components of χ are not separate Groth16 public inputs, the ACM receives them as transaction parameters in order to recompute d_χ; they are therefore visible in transaction calldata. The proof certifies the conjunction that $C(P_{\mathrm{enc}}, \ell, (a, b)) = 1$, $h = \mathsf{Com}(P_{\mathrm{enc}}, \ell; r)$, $d_\chi = \mathsf{Poseidon}(\chi)$, and the selected external-predicate witnesses satisfy their policy-bound verification relations. Before verifying π, the ACM checks that the resource is governed by the stated policy, that τ is current, that the nonce has not been used for the same requester–resource context, and that the digest recomputed from

> the transaction parameters equals the public signal d_χ. Thus, within one ACM deployment, a proof generated for one requester identifier, resource, and nonce cannot be accepted under a different tuple.

Location in the revised manuscript: Section "Privacy Evaluation," subsection "Dataset and corpus generation," opening paragraph.

> We evaluate all three objectives on one policy corpus, so the supervised attacks and the clustering cross-check see the same transactions. The corpus holds **1077 distinct policies** run as **1773 on-chain executions** on *Polygon Amoy*, each execution being one commit transaction followed by one access transaction. The policies are produced by a deterministic generator with a fixed seed (`unified-decoupled-2026`). For reproducibility, this generator uses a deterministic test issuer to create the private profile-credential witnesses used by policies that exercise the credential predicate; this testing key does not model issuer governance or production key management.

Location in the revised manuscript: Section "Privacy Evaluation," updated Table 7, "Headline supervised results on the Polygon Amoy dataset."

Table 7. Headline supervised results on the Polygon Amoy dataset (30 policy-disjoint seeds; a task fails iff $\mathrm{AUC}_{\mathrm{mean}} > 0.55$ and at least 50% of seeds are significant).

Property	Hidden target	Headline supervised result
PCH	DUO terms and ICD-10 chapters	19 of 20 content-recovery tasks pass; AUC 0.464–0.570, significant seeds median 3%. ICD ch5 fails (AUC 0.667, 53%); its separability is carried entirely by the random pad target*
PI	Related policy-pair identity	3 gating tasks pass; AUC 0.509–0.549; significant seeds median 7%, max 10%
SPP	Structure, predicate, parameter, and nesting-depth buckets	20 of 21 pairwise contrasts pass; AUC 0.468–0.571, significant seeds median 7%. `nestingDepth` medium/deep fails (AUC 0.563, 53%); carried by the pad target*

* Rerunning the same protocol on the gas minus the logged pad target returns both tasks to PASS (the PCH and SPP subsections).

Location in the revised manuscript: Section "Privacy Evaluation," updated Table 15, "Single-point cost profile of the evaluated prototype configuration."

Table 15. Single-point cost profile of the evaluated prototype configuration at DEPTH = 64. Gas, logs, calldata, and local prover timings are summarized over the 1,773 revised Polygon Amoy executions.

Metric	Value / source
Circuit size	81,740 R1CS constraints; 80,884 wires; DEPTH = 64
Public interface	4 public inputs plus 1 output
Padded on-chain gas	Commit 304,167; access 1,057,283 mean gas
Event logs	Commit: 1 log / 180 bytes; access: 2 logs / 296 bytes
Calldata length	Commit 196 bytes; access 612 bytes (constant across policies)
Local witness time	763.6 ms mean
Local prove time	2,884.2 ms mean

Comments 3: *The policy language is still quite limited. The system focuses on fixed-length Boolean RPN programs, while many realistic healthcare consent policies involve temporal constraints, revocation, obligations, delegation, purpose limitation, institutional approval, and evolv-*

ing consent. Some of these are treated as external predicate bits, but this moves complexity outside the proven statement. The paper should better explain what kinds of real consent policies can actually be supported end-to-end.

Response 3: We agree that "external predicate bit" left the guarantee unclear, and the revision answers the question in two steps: the relation no longer accepts unconstrained predicate bits, and the Design section now states feature by feature what the prototype establishes at authorization time.

In the revised relation every predicate bit is derived in-circuit from a witness bound to committed parameters: ontology-scoped conditions (disease, study design, ancestry, geography, clinical term) are Merkle-membership proofs against a provider-committed whitelist root; user-, project-, and institution-specific restrictions are membership proofs of the holder pseudonym in provider-committed allowlists; and the profile-backed data-use terms are derived from the claims of an EdDSA-Poseidon credential that binds the policy commitment, the requester pseudonym, the resource, the claims, a validity interval, and a credential identifier. These profile-backed terms prove possession of signed claims at authorization time, not that subsequent use conforms to a declared purpose. The policy's own validity window is committed and checked against the public request epoch, which the contract requires to equal the current hour.

A new table in the Design section classifies each consent feature as decided in-circuit, decided on-chain by the access-control contract from public state, committed but not decided at request time, or not provided. It also states the compiler's composition limits and distinguishes institution-specific allowlists from institutional or ethics approval, which remains an out-of-circuit step. Policy revocation and resource rebinding govern future authorizations but cannot retract data already released or enforce downstream duties.

The Conclusion lists the unsupported features among its limitations; the Design table separately states that policy revocation and resource rebinding affect future decisions only. The Discussion proposes a governed issuer registry with in-circuit membership and non-revocation witnesses, together with holder-authorized presentation through a per-request signature. These extensions could preserve the Boolean interpreter and the fixed public-signal shape, but they would require a revised circuit and new proving and verification keys.

Changes in the manuscript. The supported policy class and its composition rules are now stated explicitly in the Design section and summarized feature by feature in a new table. The Discussion distinguishes implementable extensions from current guarantees, and the Conclusion lists the unsupported features.

Location in the revised manuscript: Section "TPS-ZKPI Design," subsection "Policy Representation and Interpretation," paragraph "Scope of the supported policies."

> The policy class is deliberately restricted, and Table 4 states what the prototype establishes end-to-end. Unrestricted use (NRES) is supported only as a standalone policy; other supported terms may be combined by AND/OR in programs of at most L tokens, with each term appearing at most once and MOR/TS restricted to conjunctions. Categorical data-use terms, ontology-scoped conditions, requester allowlists, profile-backed terms, the policy validity window, and time-limited use are decided at authorization time, either inside the proved relation or by the ACM from public state; profile-backed terms establish possession of issuer-signed claims, not conformity of subsequent data use with a declared purpose. A publication moratorium is bound into the policy as an accepted obligation but cannot be decided at request time, and institutional or ethics approval, downstream purpose limitation, post-access duties, delegation, and credential revocation are left to external governance and the issuer

lifecycle. On-chain policy revocation and resource rebinding affect future authorization decisions only; they do not retract data already released or enforce cessation or deletion after access. Accordingly, the prototype is an access-time consent checker for bounded Boolean policies over standardized terms.

Location in the revised manuscript: Section "TPS-ZKPI Design," subsection "Policy Representation and Interpretation," new Table 4, "Consent-policy features and how far the prototype establishes them."

Table 4. Consent-policy features and how far the prototype establishes them. "In-circuit" means the condition is part of the proved relation; "on-chain" means the ACM enforces it from public state; "committed" means the value is bound into the policy but compliance is not decided at request time.

Consent feature	Status	What is established at authorization time
Unrestricted use (NRES)	In-circuit	Compiled as a standalone unconditional policy; it cannot be combined with another DUO term.
Categorical data-use terms combined with AND/OR	In-circuit	The committed program and its length are evaluated by the interpreter; each term may appear at most once.
Disease, study-design, ancestry, geography, and clinical-term scopes (DS, RS, POA, GS, CC)	In-circuit	Merkle membership of the requested code in a provider-committed whitelist whose root is bound in the parameter commitment.
User-, project-, and institution-specific restrictions (US, PS, IS)	In-circuit	Membership of the holder pseudonym in a provider-committed allowlist for the scope.
Profile-backed terms (GRU, HMB, NMDS, GSO, NPUNCU, NPOA, NPU, NCU)	In-circuit	EdDSA-Poseidon credential over policy commitment, pseudonym, resource, claims, validity interval, and identifier; the attributes are derived from the signed claims, but subsequent conformity with a declared purpose is not established.
Policy validity window	In-circuit + on-chain	The committed window is checked against the public request epoch, which the ACM requires to equal the current hour.
Time-limited use (TS)	On-chain	The committed duration starts a per-requester window on the first grant; later requests outside it are denied. TS may appear only conjunctively.
Publication moratorium (MOR)	Committed	The deadline is bound into the policy as an accepted obligation; publication conduct is not observable at request time. MOR may appear only conjunctively.
Prospective consent withdrawal and policy evolution	On-chain	A successor policy supersedes its predecessor, a policy can be revoked, and a resource can be rebound for future decisions; this does not retract data already released or enforce downstream cessation or deletion.
Institutional or ethics approval (IRB)	Not provided	Documentary approval and committee review remain an out-of-circuit authorization step.
Downstream purpose limitation, post-access duties (PUB, COL, RTN), delegation, credential revocation	Not provided	Carried by external governance, the data access agreement, and the issuer lifecycle, outside the proved statement.

Location in the revised manuscript: Section "Discussion," paragraphs on issuer governance and the proposed extension.

A concrete deployment can close this gap without changing the Boolean interpreter. First, a governed issuer registry would publish schema-qualified issuer keys and their validity epochs; registration would validate a canonical, subgroup-safe public key

> and require proof of possession. Second, an issuer would sign a domain-separated credential containing a holder-key commitment, claim vector, authorization scope, validity interval, and unique credential identifier. Third, the circuit would verify both the credential signature and a Merkle proof that the issuer key is active in the registry, while continuing to derive policy attributes from the signed claims. The holder would retain its secret key and authorize each request by signing the context digest and request epoch; verifying this second signature inside the circuit would prevent a TPS that observes one credential presentation from generating proofs for fresh sessions. Finally, an epoch-specific sparse-Merkle or accumulator root would support a private non-revocation witness for the credential identifier. The registry and revocation roots can occupy fixed-format public or committed parameter slots, preserving the Boolean interpreter and the shape of the policy-independent public interface; integrating these checks would nevertheless require a revised circuit and new proving and verification keys.

Location in the revised manuscript: Section "Conclusion," third paragraph.

> These findings are empirical and scoped to passive observation of commitments, proofs, public parameters, calldata, events, and gas on the generated-policy corpus. Policy confidentiality remains contingent on the TPS following its workflow, and institutional credential trust remains contingent on issuer governance. The proved policy class covers access-time checks for bounded Boolean policies over standardized terms. Outside this scope lie downstream purpose limitation, post-access duties, delegation, credential revocation, and channels involving timing, ordering, mempool activity, off-chain communication, adaptive probing, or correlations between authorization outcomes and policy classes.

Comments 4: *The privacy evaluation is too narrow. The adversary is mainly a passive metadata-only public-chain observer, and the effective leakage channel is reduced mostly to gas usage. This is useful, but it does not cover adaptive requester probing, repeated access attempts, timing/order leakage, off-chain TPS communication patterns, compromised TPS behaviour, or correlations between authorization outcomes and policy classes. The paper should either broaden the evaluation or tone down the privacy claims.*

Response 4: We chose to tone down the claims and to say precisely what the evaluation covers, while making the evaluation of the modeled channel more rigorous.

Toned-down claims. The abstract, Discussion, and Conclusion now describe the result as empirical resistance against the modeled public-chain metadata attacks, and the Discussion lists every exclusion the reviewer names—timing and ordering, mempool and broadcast behaviour, off-chain TPS communication, adaptive probing, repeated attempts, correlations between outcomes and policy classes (every request in the corpus is granted, so this correlation is not exercised), and compromised TPS behaviour—with the statement that the results should not be read as establishing general policy confidentiality against them. The threat model also states why the gas channel is wider than the verifier's arithmetic: the EVM prices calldata per byte, so any deterministic difference in how a public argument is encoded reaches the observer through gas even when calldata length is constant.

A more rigorous treatment of the modeled channel. The evaluation was re-run on the revised system and a new corpus. Two of the 45 supervised tasks now meet the failure criterion; we report them as failures and trace them, by subtracting the logged pad target from the access

gas, to coincidences of the padding's own random draws with a label (Reviewer 1, Comments 5 and 6), and we quantify the criterion's false-positive rate with a null simulation. The re-run ablation shows what the un-padded transcript leaks and why: not the policy logic, which the fixed circuit already hides, but the byte pricing of the requester pseudonym, whose encoding width in our harness correlates with whether a policy activates the profile predicate. We report this because it is the kind of channel the reviewer's broader adversary would exploit, and because it shows that the evaluation does detect sub-kilogas deterministic channels when they exist.

We did not add adaptive or timing experiments in this revision. They require a different corpus design (repeated, adversarially chosen requests against the same policy, and control over block inclusion) and an adversary model with an active component; the Conclusion names them as the next step for the evaluation.

Changes in the manuscript. The revised Abstract names the evaluated adversary, the evaluation defines the observable channel and exclusions, and the Discussion and Conclusion state the empirical scope and unmodeled channels explicitly.

Location in the revised manuscript: Abstract.

> We evaluate TPS-ZKPI on Polygon Amoy against metadata-only public-chain adversaries with supervised and unsupervised inference attacks. Of 45 supervised tasks, 43 remain below the multi-seed leakage criterion; diagnostic analysis traces the two threshold crossings, and one corresponding small clustering result, to finite-corpus coincidences in policy-independent padding targets. A paired ablation attributes suppression of the modeled gas channel to randomized padding, at an overhead of about 46% on access and 38% on commit gas.

Location in the revised manuscript: Section "Privacy Evaluation," subsection "Adversary and observables."

> Of these features, calldata length, proof size, and log counts/sizes are constant across policies by design; any policy-dependent signal can therefore only flow through the two gas measurements (`commitGasUsed`, `accessGasUsed`). The fixed circuit and the commitment-only interface make the non-gas features constant; randomized gas padding protects the gas channel that remains. That channel is wider than the verifier's arithmetic: because the EVM prices calldata per byte (four gas for a zero byte, sixteen otherwise), the gas of an access transaction also encodes the byte pattern of its public arguments, so any deterministic difference in how an argument is encoded reaches the observer through gas even when the calldata length is constant. The constant features remain in the attack pipeline but carry no variation to exploit. We do not model side channels such as block timing, transaction ordering, mempool or broadcast patterns, or off-chain TPS communication, nor the non-goals stated in the Problem and Threat Model section.

Location in the revised manuscript: Section "Discussion," opening and scope paragraphs.

> The evaluation supports a scoped claim: against the modeled public-chain metadata adversary, TPS-ZKPI provides empirical resistance to policy leakage. Of the 45 supervised tasks, 43 stay below the multi-seed leakage criterion, and unsupervised clustering shows no recoverable structure beyond one small, permutation-significant nesting-depth ARI. The two tasks that meet the criterion—one ICD-10 chapter and the `nestingDepth medium/deep` contrast—are reported as failures, and both are

> carried by the random pad target rather than by the policy: subtracting the logged target from the access gas returns each to PASS, and replacing the gas features by policy-independent draws reproduces such failures in four of ten replicates. This is a property of the criterion on a corpus in which coverage-layer policies are executed once, so the thirty seeds resample the same random draws; we keep the criterion as stated and document the diagnostic, since lowering the significance threshold or raising the AUC threshold to absorb these cases would weaken it for genuine leaks. The gas-padding ablation attributes the gas-channel protection to randomized gas padding: without it the same attacks recover the policy family with AUCs up to 0.90. Content hiding itself rests on the commitment-only interface and the content-independent transcript normalization; padding is the defense-in-depth layer that absorbs whatever deterministic gas differences remain, of which the ablation exhibits one that does not come from the policy logic at all but from the byte encoding of a public transaction argument. Together with fixed-length policy representation, reusable interpretation, and commitment binding, these mechanisms limit the extent to which public-chain observers infer hidden policy content or structural properties from authorization transactions.
>
> The scope of this claim is the adversary of Section 6.1: a passive observer of commitments, proofs, public parameters, calldata, events, and gas, on a corpus of generated policies. Timing and ordering of transactions, mempool and broadcast behaviour, off-chain communication with the TPS, adaptive requester probing, repeated access attempts, correlations between authorization outcomes and policy classes (every request in the corpus is granted), and compromised TPS behaviour are not modeled, and the results should not be read as establishing general policy confidentiality against them.

Location in the revised manuscript: Section "Conclusion," limitations and future-directions paragraphs.

> These findings are empirical and scoped to passive observation of commitments, proofs, public parameters, calldata, events, and gas on the generated-policy corpus. Policy confidentiality remains contingent on the TPS following its workflow, and institutional credential trust remains contingent on issuer governance. The proved policy class covers access-time checks for bounded Boolean policies over standardized terms. Outside this scope lie downstream purpose limitation, post-access duties, delegation, credential revocation, and channels involving timing, ordering, mempool activity, off-chain communication, adaptive probing, or correlations between authorization outcomes and policy classes.
>
> The next steps are therefore clear: reduced-trust proof generation and holder-signed request contexts can reduce the TPS trust concentration; governed issuer registries with in-circuit membership and revocation witnesses can strengthen credential provenance; and evaluation against active adversaries can test transcript normalization in the timing and ordering channels beyond the present model.

Comments 5: *The comparison with related work should be strengthened by considering recent blockchain-based access-control architectures that combine smart contracts, SSI-oriented trust models, and cryptographic mechanisms for controlling access to sensitive claims. In particular, the paper should better position TPS-ZKPI with respect to the issues discussed in DOI: 10.1016/j.bcra.2024.100196, especially regarding verifier authorization, smart-contract accountability, off-chain trust assumptions, and practical access-control workflows.*

Response 5: We thank the reviewer for the pointer. The Related Work section now contains a paragraph on self-sovereign-identity architectures that positions TPS-ZKPI against Buccafurri, De Angelis, and Nardone (Blockchain: Research and Applications, 2024) along the four dimensions named, together with recent zero-knowledge authentication work (Madine et al., 2025) and surveys of zero-knowledge proofs for blockchain privacy.

- *Verifier authorization.* In the CP-ABPRE design the holder's claims are readable only by verifiers that satisfy a holder-defined policy. In TPS-ZKPI the protected object is the provider's consent policy, not the requester's claim set: the claims are consumed as private witnesses of a proof whose statement is fixed and whose verifier is a public contract, so any observer may verify and verifier authorization is not a design question.
- *Smart-contract accountability.* There the contract meters the release of claims and keeps verifiers accountable; here the contract records the context-bound authorization outcome, and accountability attaches to that record rather than to claim release.
- *Off-chain trust.* An SSI verifier that decrypts a claim learns it; the TPS learns both policy and claims. This is why the Problem section treats the TPS as an explicit trust boundary and the new threat/guarantee table states what survives its compromise (Comment 1).
- *Access-control workflow.* The two designs sit on opposite sides of the same authorization relation. Combining them through holder-authorized presentation, including a per-request signature over the authorization context, would prevent a TPS that observes one credential presentation from reusing the evidence for fresh sessions.

Changes in the manuscript. The complete comparison paragraph and the added reference are reproduced below.

Location in the revised manuscript: Section "Related Work," new paragraph on self-sovereign-identity architectures.

> Self-sovereign-identity (SSI) architectures move authorization to verifiable claims held by the requester and shift the question to whom the holder may trust with those claims. Buccafurri et al. combine smart contracts with ciphertext-policy attribute-based proxy re-encryption so that a holder's claims can be read only by verifiers that satisfy a holder-defined policy, with the contract keeping verifiers accountable for the access they obtain; zero-knowledge presentations of credentials likewise let a patient authenticate anonymously against public or private chains, and surveys of zero-knowledge proofs for blockchain privacy catalogue this design space. TPS-ZKPI sits on the other side of the same authorization relation. In SSI designs the protected object is the requester's claim set and the policy that governs it is the holder's, whereas here the protected object is the provider's consent policy: the requester's claims are consumed as private witnesses of a proof whose statement is fixed and whose verifier is a public contract, so verifier authorization is not a design question (any observer may verify) and accountability is carried by the recorded, context-bound outcome rather than by a contract that meters the release of claims. The trust the two settings place off-chain also differs: an SSI verifier that decrypts a claim learns it, whereas the TPS learns both policy and claims, which is why Section 4 treats the TPS as an explicit trust boundary and Table 3 states what survives its compromise. Combining the two—holder-authorized presentation of the claims that TPS-ZKPI consumes as witnesses, including a per-request signature over the authorization context—would prevent a TPS that observes one presentation from reusing the evidence for fresh sessions, as discussed in Section 7.

Location in the revised manuscript: Reference list, new entry for Buccafurri et al. (2024).

> Buccafurri, F.; De Angelis, V.; Nardone, R. How can the holder trust the verifier? A CP-ABPRE-based solution to control the access to claims in a Self-Sovereign-Identity scenario. *Blockchain: Research and Applications* 2024, 5, 100196. `https://doi.org/10.1016/j.bcra.2024.100196`.

Comments 6: *The cost and scalability analysis needs more depth. The reported gas-padding overhead is substantial, around 38% for access and 40% for commit operations, and the evaluation is performed at a single circuit depth. The paper should evaluate how costs change with larger policy programs, richer predicates, more complex DUO/ICD conditions, and more realistic access workloads. Otherwise, it is difficult to judge whether the design is practical for real healthcare data-sharing platforms.*

Response 6: We added two reproducible offline scalability experiments and a scenario-based workload analysis. No new Amoy data were collected and the frozen RC1 circuit, proving key, and verifier semantics were not modified.

Program and ontology capacity. For program lengths 32, 64, and 128 (with eight Merkle levels), constraints are 64,945, 81,740, and 134,009; median proving times are 0.411, 0.521, and 0.734 s. For Merkle depths 4, 8, 12, and 16 (with $\mathsf{DEPTH} = 64$), constraints are 73,876, 81,740, 89,604, and 97,468; median proving times are 0.512, 0.499, 0.552, and 0.571 s. Each row contains 30 verified proofs and reports constraints, wires, WASM/zkey size, and witness/prove/verify median, p95, and IQR. Every variant retains five public signals, so the verifier interface is unchanged.

Policy and DUO complexity in the production circuit. We additionally ran 30 verified proofs each for: Simple (`DS`); Ontology-heavy (five ontology slots); Profile-heavy (multiple profile-backed DUO terms); and Max-mixed (all five ontology categories, profile terms, `US/PS/IS`, `MOR`, and `TS`). The programs range from 3 to 61 tokens and activate one to all 11 production slots, but their median witness times remain 0.335–0.354 s and median proving times 0.484–0.545 s. Profile-backed cases have about 0.6 s of additional end-to-end evidence preparation; the fixed-circuit witness/prove/verify work remains stable. We did not directly sweep M because only $N = 13, M = 11$ activates the implemented production slot masks; other M values use a generic verifier branch and would not be a like-for-like scalability comparison.

Access workloads. At concurrency 1, 2, and 4, we executed five warm-ups and then 100 verified requests per level, rotating the four policy classes. The measured throughputs are 0.805, 0.787, and 0.936 proofs/s; the best observed single-host result is therefore 0.936 proofs/s. At 1,000, 10,000, and 100,000 accesses/day, the average loads are 0.012, 0.116, and 1.157 requests/s, requiring one, one, and two equivalent prover instances when retaining 50% headroom. Using the measured 1,056,674 gas/access at 30 gwei, these scenarios consume 31.70, 317.00, and 3170.02 POL/day. One commit is 304,098 gas (0.009 POL) and is stated separately; we do not invent a commit/access ratio. The manuscript labels all workload rows as scenarios rather than measurements of hospital traffic or public-chain throughput.

Changes in the manuscript. New Table 16 reports both capacity sweeps; new Table 17 reports fixed-circuit policy complexity, TPS load, and daily-volume scenarios. The Ablation subsection replaces the former theoretical M statement with these measurements and the fairness rationale for not sweeping M. The Discussion reports the observed throughput boundary and cost scenarios without extending the production-availability claim.

Location in the revised manuscript: Section "Privacy Evaluation," subsection "Ablation," and new Tables 16–17.

> Table 16 reports two offline capacity sweeps, each with 30 verified proofs per configuration on an Apple M1 using rapidsnark. With $N = 13$, $M = 11$, and eight Merkle levels fixed, increasing DEPTH from 32 to 128 raises the circuit from 64,945 to 134,009 constraints and the median proving time from 0.411 to 0.734 s. With DEPTH $= 64$ fixed, increasing the ontology-tree depth from 4 to 16 raises the circuit from 73,876 to 97,468 constraints and the median proving time from 0.512 to 0.571 s.
>
> We deliberately do not sweep M: only the production configuration $N = 13, M = 11$ activates the implemented production slot masks, whereas other M values enter a generic verifier branch and would not be a like-for-like comparison. Across the four fixed-circuit policy classes, median witness time is 0.335–0.354 s and median proving time is 0.484–0.545 s.

Location in the revised manuscript: Section "Discussion," paragraph "The cost of padding."

> The offline load benchmark rotates the four policy classes through 100 verified requests after five warm-ups at each concurrency level. On the evaluated Apple M1, the best observed batch throughput is 0.936 proofs/s at concurrency four. At 1,000, 10,000, and 100,000 accesses/day, retaining 50% headroom requires one, one, and two equivalent prover instances. These are scenario-based calculations, not hospital-traffic measurements or public-chain throughput claims.